

ReAct and ART Prompting Methods

ReAct Prompting

ReAct (Reason+Act) is a few-shot prompting paradigm that has the LLM alternate between **verbal reasoning** and **external actions**. In practice, prompts include example question-solving *trajectories* where the model emits a sequence of **Thought** and **Action** steps, each followed by an **Observation** from a tool or environment. The LLM uses each Observation to inform its next Thought, forming a loop of reasoning and interaction ¹ ². This interleaved process allows the model to ground its chain-of-thought in real-world data (e.g. search results) and refine its plan dynamically ¹ ².

ReAct prompts instruct the model to alternate “Thought” (reasoning) and “Action” steps, retrieving information as needed. In the example above (from Yao et al.), the model generates a Thought, issues a Search action, incorporates the observation, and iterates until answering the question ¹ ².

1. **Prepare few-shot exemplars:** Build a prompt with 1–3 example Q&A trajectories. Each example consists of a **Question** and interleaved steps labeled “Thought 1”, “Action 1”, “Observation 1”, “Thought 2”, etc. Observations record the output of each action (e.g. a search result) ³ ².
2. **Prompt the target question:** Append the new question in the same format (usually without examples for zero-shot, or after examples for few-shot). The model is prompted to continue the pattern (Thought, Action, Observation...) until it reaches a final **Answer**.
3. **Execute actions iteratively:** Whenever the model outputs an *Action* (e.g. `Search[...]` or `Calculator[...]`), run that tool externally and feed the result back to the model as the corresponding *Observation*. The LLM then uses this observation in the next Thought step ¹ ².
4. **Iterate to completion:** The model continues alternating Thought and Action steps. For knowledge-intensive tasks (like multi-hop QA), include several cycles of Thought/Action/Observation. For action-heavy tasks (like a text-based game), the model may insert Thoughts more sparsely (only when needed) ⁴. Stop when the model produces a final answer or a “Finish” action.
5. **Key Strengths:** ReAct combines chain-of-thought with tool use, improving factual accuracy. It often **outperforms “Act-only” or vanilla CoT prompting** on QA and decision tasks ⁵ ⁶. Its output is human-readable (the Thoughts explain the reasoning) which aids interpretability ¹ ². Because ReAct can call up-to-date tools (search, APIs, calculators) on-the-fly, it avoids hallucinating facts.
6. **Flexibility:** The prompt format is intuitive – one simply writes out the reasoning next to each action. No special fine-tuning is needed ⁷. It works across diverse tasks as long as you provide relevant examples ⁷ ³.
7. **Differences vs. ART:** ReAct relies on manually crafted in-context examples for each task and expects the model to decide actions as it goes. In contrast, ART automatically selects examples from a broad library and outputs a fully structured “program” of steps with explicit pauses for tools ⁸ ⁹.

8. **Cautions:** Success depends on well-chosen examples and reliable tools. If a search result is irrelevant, ReAct can go off-track ¹⁰. ReAct's rigid format (Thought→Action→Observation) also means it may be less flexible than unconstrained CoT.

ReAct Prompt Template Examples: Use the Thought/Action/Observation pattern. Below are two sample templates (placeholders in brackets):

```
Question: [Your question here]
Thought 1: [Initial reasoning about how to solve the question]
Action 1: Search[... or Calculator[...]]
Observation 1: [Result returned by the tool or environment]
Thought 2: [Use Observation 1 to refine reasoning]
Action 2: [Another tool call if needed]
Observation 2: [Result of Action 2]
...
Answer: [Final answer derived from the reasoning and observations]
```

```
Question: What is the result of 17 * 13 plus 10?
Thought 1: First compute 17 * 13.
Action 1: Calculator[17 * 13]
Observation 1: 221
Thought 2: Now add 10 to 221.
Action 2: Calculator[221 + 10]
Observation 2: 231
Answer: 231
```

Automatic Reasoning and Tool-use (ART)

ART is an automated CoT+tools framework that treats reasoning steps as a program ⁸ ⁹. It maintains a **library of example tasks and solutions**, and when given a new problem, it finds similar multi-step examples to guide the LLM. The prompt instructs the model to output a numbered list of solution steps. Crucially, ART “pauses” the model whenever a step requires a tool: the system runs the tool (search, calculator, code execution, etc.) and feeds the result back before continuing generation ⁸ ¹¹. This ensures correct integration of external computations. ART thus combines planning and tool calls in a single coherent workflow.

The ART workflow: the model uses a task library of examples to compose a step-by-step “program.” When a tool call is generated, the process halts, the tool is run, and the output is added before resuming ⁸ ⁹.

1. **Build example libraries:** Create a **Task Library** of solved problems (with step-by-step solutions) across relevant domains (e.g. math, search, coding) and a **Tool Library** (e.g. search engine, code executor) ¹². Each example solution is written in a structured format (like pseudo-code or ordered steps).
2. **Select demonstrations:** For a new task, prompt the LLM with a few relevant examples from the Task Library. Each example shows how to break a problem into steps and when to use tools ⁸ ¹³.

3. **Generate step-by-step solution:** Ask the model to produce a numbered sequence of steps to solve the task. It should explicitly indicate when to invoke a tool (e.g. "Use search for X"). Each step is like a line in a program ⁸ ¹⁴.
4. **Tool invocation and continuation:** When the model's output includes a tool action, the system pauses and executes that action. For example, if Step 2 is `Search["population of France"]`, the system performs the search and appends the result as part of the step. After inserting the tool output, resume the model so it can continue from the updated context ¹⁵ ¹⁴.
5. **Complete answer:** Continue until the sequence yields a final answer. The model's steps and computed results together form a clear "program" of the reasoning. Humans can optionally review or correct any step before finalizing the answer ¹⁶ ¹⁷.
6. **Key Strengths:** ART generalizes across tasks with **zero-shot planning** from the example library ⁸ ⁹. It has shown large gains on benchmarks: e.g. +15 percentage points on known tasks and +7 points on unseen tasks versus naive few-shot ¹⁸ ¹⁹. In many cases it matches or outperforms carefully engineered CoT prompts ¹⁹ ¹⁸. Its structured output and pause-resume mechanism ensure tool calls are handled reliably.
7. **Flexibility & Extensibility:** New tools or domains are easy to add by updating the libraries ¹⁷ ¹⁶. Since the steps are written in a program-like format, developers can inspect and correct mistakes on-the-fly and immediately re-run the solution ¹⁶ ²⁰. This makes ART highly adaptable across math, code, search, and other categories ²⁰.
8. **Differences vs. ReAct:** ART **automates prompt design** by selecting and stitching examples, whereas ReAct requires hand-crafted examples per task ⁸ ³. ART treats reasoning as an explicit program, often yielding more systematic multi-step plans, while ReAct is more free-form. ART excels at tasks needing exact computation or code (thanks to built-in tools), whereas ReAct is simpler to apply for text-based QA with searches.
9. **Cautions:** Because ART relies on correct step generation, errors can cascade if an early step is wrong ²¹. Its gains depend on having a good library of examples. Code-generation steps may fail if the model's code is imperfect. Still, these are partly mitigated by the ability to validate and refine each step ²¹ ¹⁶.

ART Prompt Template Examples: Below are two illustrative prompts. Each shows example tasks with solutions, then the target task.

Task: "Find the population of Spain and its capital city."

Solution Steps:

1. Search["population of Spain"] → "47 million".

2. Search["capital of Spain"] → "Madrid".

Answer: "47 million, Madrid".

Task: "Find the population of France and its capital city."

Solution Steps:

1. Search["population of France"] → "67 million".

2. Search["capital of France"] → "Paris".

Answer: "67 million, Paris".

Task: "Calculate $(7 * 6) + 15$."

Solution Steps:

1. Compute $7 * 6 = 42$.

2. Compute $42 + 15 = 57$.

Answer: 57.

Task: "Calculate $(8 * 9) + 12$."

Solution Steps:

1. Compute $8 * 9 = 72$.

2. Compute $72 + 12 = 84$.

Answer: 84.

Each prompt shows solved examples in a clear step-by-step format, then asks the LLM to solve a new task similarly. This guides the model to output a structured plan, using tools or calculations exactly where needed.

Sources: Core definitions and workflows for ReAct ¹ ² and ART ⁸ ⁹ are drawn from the original publications. Performance and usage insights are based on reported results ⁵ ¹⁸ ¹⁹, and prompting guides ⁹ ¹³.

¹ ⁶ ⁷ [2210.03629] \model: Synergizing Reasoning and Acting in Language Models

<https://arxiv.org/html/2210.03629>

² ³ ⁴ ⁵ ¹⁰ ReAct Prompting | Prompt Engineering Guide

<https://www.promptingguide.ai/techniques/react>

⁸ ¹⁵ ¹⁶ ¹⁹ [2303.09014] ART: Automatic multi-step reasoning and tool-use for large language models

<https://arxiv.org/abs/2303.09014>

⁹ ¹¹ ¹⁷ Automatic Reasoning and Tool-use (ART) | Prompt Engineering Guide

<https://www.promptingguide.ai/techniques/art>

¹² ¹³ ¹⁴ ¹⁸ ²⁰ ²¹ ART: Automatic Multi-step Reasoning and Tool-use for Large Language Models

<https://blog.synapticlabs.ai/art-automatic-multi-step-reasoning-and-tool-use-for-large-language-models>